

## Exercise 1

23.05.2023

---

### General Information

This exercise will guide you through various surface EMG data postprocessing and analysis steps. We suggest using MATLAB to visualize, quantify and process the EMG data and corresponding force signal during the exercises. Note that using Python is also possible, although we will use MATLAB for the exemplary solution. If you have not installed MATLAB yet, click on the link below and take advantage of FAU's Campus-Wide License to install and use MATLAB on your computer: [www.mathworks.com/products/matlab/student.html](http://www.mathworks.com/products/matlab/student.html)

In addition to the base MATLAB version, you might need the MATLAB signal processing toolbox (depends on the functions you want to use). You can install it in MATLAB under Home -> Add-Ons.

MATLAB provides a lot of built-in functions that can make your life easier during these exercises. It is always a good idea to check the **MATLAB documentation**. Don't hesitate to also use Google or **write in the StudOn forum**. Also check the **Tips** section on the last page.

#### Submission:

1. Your code as a .m (or .py) file (Your main script as well as any potential self-defined functions).
2. A .pdf file with your results. The results that should be included in the report are **written in this colour** (You can save the plots from Matlab as a .png with *save as* in the plot window, or copy & paste as a .svg with *edit->copy figure*). For questions that require written text as an answer, 1-3 sentences or bullet points should usually be enough. Plots need to contain labelled axes and suitable units.

- 
- Please **do not** submit a document with both the code **and** the plots & answers! Only submissions with two **separate** files (a .pdf with the plots & answers, and a separate .m/.py/.ipynb file containing your executable code) are valid.
  - Plots need to contain labelled axes and suitable units.

Submission Deadline: 19.06.2023, 20:00 pm

Submit your results by sending the required files via mail to **marius.osswald@fau.de**.

## The Experiment

In this laboratory, you will investigate how your motor cortex controls your movements. You will analyse certain features of the High-Density Surface-EMG signals and the behaviour of the motor units during a variety of movements of your index finger while recording the muscle activation signals from your *first dorsal interosseus* muscle (see Fig. 1 A), responsible for abduction and flexion of the index digit.

The experimental protocol consists of isometric contractions in three distinct degrees of freedom (DoFs): Index digit abduction, flexion and extension. The subjects will follow force feedback ramps of two different force levels (see Fig. 1 B). A force sensor will measure the force, which is displayed on a screen in real time to give the subject feedback on the currently applied force.

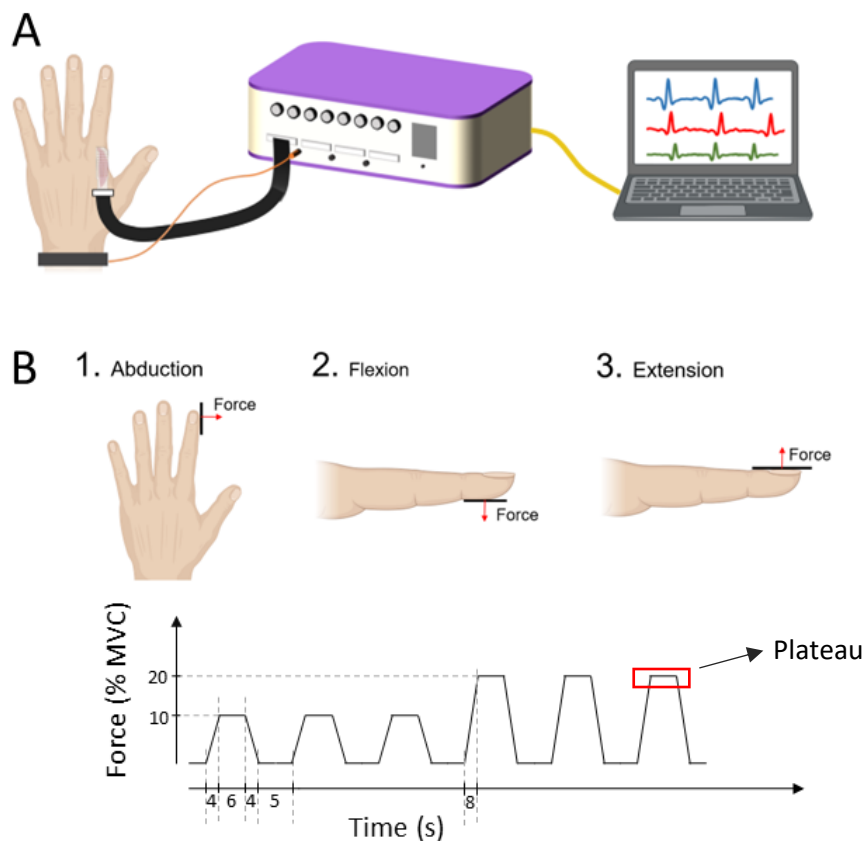


Figure 1. A: Experimental HD-sEMG Setup. B: Experimental protocol, showing the three contractions in the distinct DoFs and the force feedback ramps.

Along with this document, you receive the files you recorded during your experiment session, as well as files containing the decomposed motor unit data of one exemplary group. For [Task 1. Force Data](#) and [Task 2. EMG Data](#), use the data contained in your groups folder. For [Task 3. Motor Unit Analysis](#), use the data in the folder *Motor Unit Data*.

## Task 1. Force Data

The data files for Task 1 and Task 2 contain the following variables:

- **EMG:** The raw EMG signals of all 64 channels
- **AUX1:** The force signal measured by the force sensor in mV
- **fsamp:** Sampling frequency

### 1.1. Conversion

Convert the force transducer signal to Newtons using the device-specific conversion factor.

$$Force\ in\ N = \frac{Force\ in\ mV}{ConversionFactor} * Gravity \quad ConversionFactor = 100\ mV ; Gravity = 9.81\ m/s^2$$

Also remove the offset that might be present in the force signals. In the phases between the ramps, the force should have a value around 0 N.

### 1.2. Visualisation

**Plot the force signals over time in seconds of all files in a suitable way.** Remember the axis labels. Check the tips section for hints on using the *subplot* command in MATLAB.

### 1.3. Filtering

In the force data, you can see a high frequency noise throughout the entire signal due to the resolution of the force sensor, that is particularly visible in the present signals because of the relatively low force values. Apply a lowpass filter to the force signal with a cutoff frequency at 100 Hz and **visualize the resulting filtered signal.**

### 1.4. Coefficient of Variation

The Coefficient of Variation (CoV) is a measure of force steadiness, indicating how much the applied force fluctuated around the desired force level. Compute the Coefficient of Variation (CoV) of the force signal during the **plateau phases** (see Fig. 1 B) of all force signals at the different force levels and DoFs. Compare the CoV values of the different force levels and DoFs. **Report your results and visualize your findings.**

The Coefficient of Variation is defined as the signals standard deviation divided by the mean:

$$CoV = \frac{Standard\ Deviation\ \sigma}{mean\ \bar{x}}$$

## Task 2. EMG Data

### 2.1. Preprocessing & Visualisation

The usual first step of any EMG signal analysis is the preprocessing of the data.

- Plot all channels of your signals in. Visually inspect the data and remove channels showing outlier behaviour for each file (flat/dead signal abnormally high spikes, movement artifacts, etc.). It is advised to rearrange your EMG data first as a matrix of shape *channels x samples* for all further analysis. **Plot all remaining channels of one exemplary file in a single figure.**

EMG signals usually contain a power line interference noise at the 50 Hz frequency.

- Filter all EMG channels of all files with a notch filter or bandstop filter, that removes the 50 Hz frequency component. **Plot the power spectrum (Frequency power in dB over frequency in Hz) of one exemplary EMG channel before and after the filtering.**

## 2.2. Root-Mean-Square (RMS) (with force)

The RMS is an important feature in EMG signal analysis that provides insight into the amplitude of the physiological activity by giving a measure of the signal power. It is calculated as the square root of the signal mean in a moving window. As you compute the square root, this also rectifies the signal.

- 4.1. Compute the average EMG across all 64 channels. You should obtain a 1D array of shape  $1 \times \text{samples}$ .
- 4.2. Compute the RMS of this average as a moving average with a window length of 200 ms. Perform this for the EMG data of all three datasets. **Visualize the result.**
- 4.3. Visualize the correlation between the resulting RMS and the respective force signal by **plotting the normalized RMS (x-axis) against the normalized Force (y-axis)** in the form of a scatter plot for all three datasets. Then quantify the correlation by computing and **reporting the correlation coefficient R.**

## Task 3. Motor Unit Analysis

For the motor unit analysis part, use the data inside the folder *Motor Unit Data*. For the raw EMG data, use the data stored in the *SIG* files for the upcoming tasks, not anymore the raw EMG data you used for the previous analysis. The data files contain the following variables:

- **SIG:** The raw EMG signals of all 64 channels
- **ref\_signal:** The force signal measured by the force sensor in mV
- **MUPulses:** The motor unit firings. One array of firings for each motor unit. The arrays contain the samples at which a firing was detected.
- **discardChannelsVec:** List of all channels that have been discarded by the decomposition algorithm due to bad signal quality or outlier behaviour
- **fsamp:** Sampling frequency
- **SIGlength:** Length of the signal in seconds

### 3.1. Spike trains (with force)

Plotting the MU action potential discharge timings as so-called **spike trains** is a good way of visualizing the firing patterns and discharge rates of different motor units during isometric or kinetic contractions (see Fig.1).

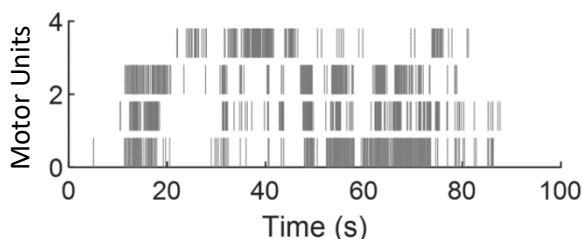


Figure 2: Exemplary MU Spike Trains. From: Del Vecchio et al. (2020).

For plotting the MU spike trains, you can use the provided MATLAB function `plotSpikeRaster(firingMatrix, 'PlotType', 'vertline', 'VertSpikeHeight', 0.9)`. The function requires a matrix `firingMatrix` with the so-called binary code of MU firings. Rows correspond to the MUs and columns to the samples (E.g. a file with 5 MUs and signal length 2000 samples, the function requires a matrix of shape 5x2000), with zeros at the positions where the MU was not active, and ones where the MU discharged an action potential. E.g., for MU1 with action potentials at samples 5, 8 and 17 and MU2 at samples 3 and 12:

```
firingMatrix=[0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 1;  
              0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
```

This matrix needs to be of data type **logical** (In MATLAB, you can convert an array or matrix of type **double** to an array of type **logical** with the command `logical(matrix)`).

**Create one figure per dataset with the spike trains of all motor units and the reference force signal in N in the same plot. Rearrange the order of the motor units in a way so the different force recruitment thresholds are visible.**

### 3.2. Firing Rate

Apart from the recruitment threshold, the firing rate / firing frequency of a motor unit is the second mechanism of the neuromuscular system to modulate force output.

For the motor units that show activation in **both** force levels, Check if there is a difference in the average firing rates during the lower force, compared to during the higher force (Only consider firings happening during the plateau phases of the force application). **Report and/or visualize the result in a suitable way.**

### 3.3. STA

Spike triggered averaging (STA) is a useful event detection technique that is often used in neural signal processing. STA is a measure to relate the spike trains to a continuous signal, in our case the raw EMG signal. It represents the average EMG signal present inside a window around the times of spike occurrences and is therefore equivalent to the cross-correlation between the EMG signal and the spike train. As a result, the STA gives us the **average motor unit action potential (MUAP) shape** of a motor unit for each channel of our EMG data. Figure 2 illustrates the concept of STA with the spike train of an exemplary Motor Unit #12.

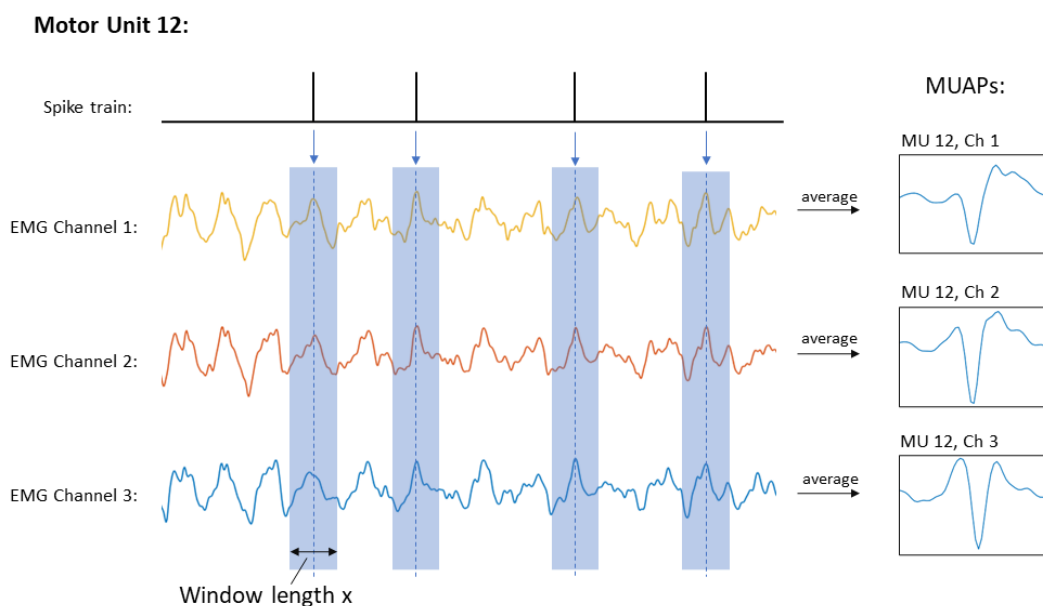


Figure 3. Concept of Spike triggered averaging. STA results in the MUAPs for each channel of the respective motor unit.

Compute the spike triggered average for all motor units of each dataset (Tip: Write a function where you give the MU spike trains of a motor unit, the raw EMG signal and window length as input parameters, that returns the STA result of all 64 (or less, if you deleted outlier channels) channels in a matrix). **Plot the MUAP shapes of the channel with the highest peak-to-peak value for each motor unit. Write 2-4 sentences of what the knowledge about the MUAP shape of each individual motor unit for each channel can be used for.**

**Note:** The resulting function for STA computation can be quite complex, as it will usually require multiple nested for-loops. It is helpful to carefully think about the necessary steps for this computation before you start coding.

## Tips

- **Script Structure:** To keep your script well-structured and clear, you can create script sections by writing `%%`, followed by a blank space and a section title. You can execute single script sections with the `Run Section` button in the toolbar or `Ctrl+Enter`, instead of having to execute the entire script. ([https://www.mathworks.com/help/matlab/matlab\\_prog/create-and-run-sections.html](https://www.mathworks.com/help/matlab/matlab_prog/create-and-run-sections.html))
- **Data Storage:** When loading the dataset into the workspace using `datasetX = load(filepath)`, the variables are combined into a structure array (`struct`), which is a very convenient way of storing variables that belong to the same dataset, as they can be easily accessed via dot notation as in e.g.: `datasetX.SIG{row,col}`. This keeps the workspace clean when working with multiple datasets, and also allows to load datasets that contain variables with the same name, as those variables would overwrite each other otherwise. You can add more fields to the `struct` by using the same dot notation, e.g. if you want to add the a new data variable `SIGfilt`: `datasetX.SIGfilt=yourFilterfunction(datasetX.SIG,...)`  
(<https://www.mathworks.com/help/matlab/ref/struct.html>)
- **Custom Functions:** If you need to apply the same data processing steps to multiple sets of data, is it advised to write a custom function for these steps, that you can then pass the datasets to. A function can either be saved in a separate `.m` file or defined at the bottom of your main script. Check <https://www.mathworks.com/help/matlab/ref/function.html> on how to define functions.
- **Data Plotting:** If you want to plot multiple related figures into one figure window, check out the `subplot` command. If you want to plot multiple plots into the same figure, use the `hold on` command. `yyaxis right` lets you plot multiple data series into one plot with independent y axes.  
(<https://www.mathworks.com/help/matlab/ref/subplot.html>)  
(<https://www.mathworks.com/help/matlab/ref/hold.html>)  
(<https://www.mathworks.com/help/matlab/ref/yyaxis.html>)  
**Important:** Don't forget to label your axes in your plots and to give them informative titles.
- If you do not know how to code a certain task, there is probably a **built-in MATLAB function** that can help you. Check the documentation or Google.